DS201: Statistical Programming

Assignment 1: Applied Probability & Statistics

Instructor: Dr. Anil Kumar Sao

Deadline: 19 Jan 2026 , 11:59 PM

Mission Briefing

Welcome, Data Scientist. You have been hired by **FutureTech Industries** to solve three critical challenges in their product lines. Your toolkit for this mission includes Probability Theory, Information Theory, and Statistical Analysis.

Submission Guidelines:

- Submit your solution as a **Jupyter Notebook** or **MATLAB** file.
- **File Naming:** `RollNumber_FirstName_Asg1` (e.g., `B21001_Rahul_Asg1`).
- **Reporting:** Include Introduction, Methodology, Results, and Conclusion in your notebook markdown.
- **Code Quality:** Comment your code. Plagiarism will be penalized.

---

1 Project: SmartKey AI (Text Analytics)

Scenario: The Intelligent Keyboard

You are the lead engineer for "SmartKey", a next-generation mobile keyboard app. To improve typing speed and implementation of "Auto-Correct", you need to understand the statistical properties of the English language.

Resources: You are provided with two large text corpora: `corpus_train.txt` (formerly fileA) and `corpus_test.txt` (formerly fileB).
Note: Pre-process the text by removing special characters, punctuation, and converting to lowercase.

Task A: Key Layout Optimization (Character Frequency)
Design an ergonomic keyboard layout by finding the most used keys.

1. Analyze `corpus_train.txt`.
2. Determine the probability of occurrence for each of the 26 English alphabets.
3. List the **Top 10** most frequent alphabets.

Task B: Data Compression (Entropy Analysis)
To sync user data efficiently, we need to compress text.

1. Calculate the **Entropy** of the English language using `corpus_test.txt`.

$$H = -\sum p_i \log_2(p_i)$$

where p_i is the probability of the i-th character.

2. What does this value tell you about the unpredictability of the language?

Task C: Speed Run Compression (Huffman Coding Challenge)

Let's turn theory into a fun optimization game — can you beat the keyboard at saving space and restoring text perfectly?

1. Using the character probabilities obtained earlier, construct a **Huffman Tree**.
2. Generate optimal **binary Huffman codes** for each character (Encoding).
3. Encode a sample text from `corpus_test.txt` using the generated Huffman codes.
4. Perform **Huffman Decoding** on the encoded bitstream and verify that the original text is perfectly reconstructed.
5. Compute the **average code length** and compare it with the entropy calculated in Task B.
6. **V**isualize the Huffman tree and explain why frequent characters get shorter codes and how decoding remains unambiguous.

Task D: Predictive Typing (Word Frequency)

The "Next Word Prediction" engine needs a vocabulary.

1. Repeat the analysis in Task A and B, but treating **Words** as events instead of characters.
2. Use `corpus_extra1.txt` (fileC) and `corpus_extra2.txt` (fileD) for this analysis.

2 Project: "Loot Box" Audit (Law of Large Numbers)

Scenario: Fair Play Investigation

A popular video game, "Galaxy Quest", uses a Random Number Generator (RNG) to drop loot from chests. Players suspect the system is rigged. The gaming commission has hired you to audit the RNG.

The Experiment: The RNG produces values between 0 and 1 (Uniform Distribution).

1. Generate n random numbers.
2. Calculate their **Mean** and **Variance**.
3. Observe how these statistics change as you increase n (from small to very large samples).
4. **Question:** Does the randomness settle into a predictable pattern? Explain why this happens (Hint: Law of Large Numbers).

Good Luck. The Future Depends on Your Analysis.